FIAP - Faculdade de Informática e Administração Paulista

Fernanda Kaory Saito – RM 551104

João Pedro Borsato - RM 550294

Maria Fernanda Camargo - RM 97956

Pedro Lucas Nunes - RM 550366

Vinicius Bernardino – RM97888

CHECKPOINT 5

Devsecops

São Paulo, 2026

Introdução

Este documento apresenta o desenvolvimento do Checkpoint 05 da disciplina de DevSecOps and Cloud Computing, com objetivo de integrar verificações automatizadas de segurança ao processo de implantação de uma aplicação utilizando containers. Para a atividade, foi reutilizado o arquivo HTML do checkpoint anterior, disponibilizado em um container Docker com Nginx.

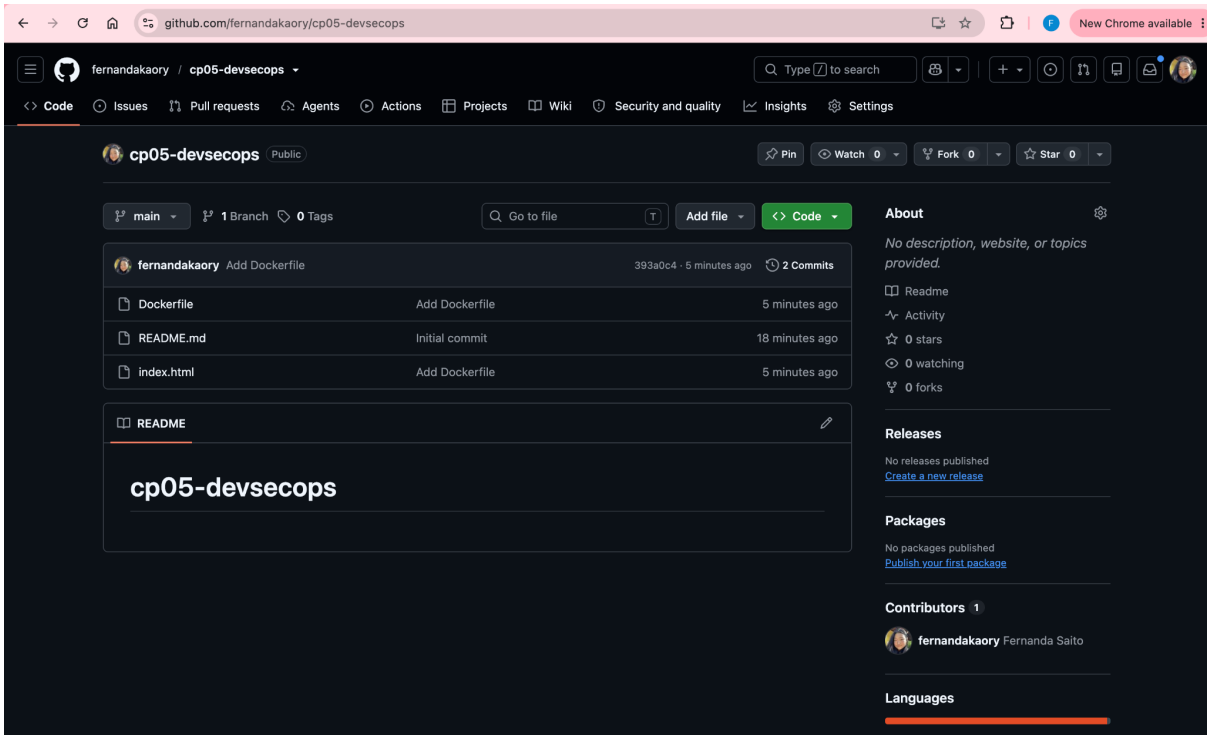
O projeto consiste em configurar um pipeline no GitHub Actions capaz de analisar credenciais expostas, padrões inseguros no código e vulnerabilidades conhecidas na imagem Docker. Além da configuração das ferramentas, foram realizados testes controlados para demonstrar seu funcionamento, acompanhados de evidências das execuções.

Estrutura da solução

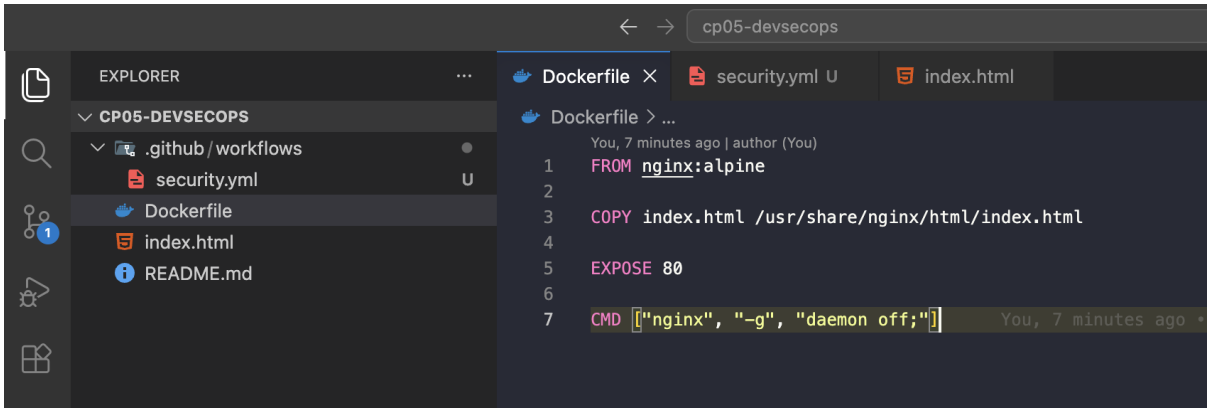
O projeto reúne o arquivo index.html dosite, um Dockerfile para construir a imagem arcanjosa:v1 e o workflow .github/workflows/security.yml. O pipeline é acionado por push e pull request e executa três verificações: Gitleaks para detecção de segredos no repositório, Semgrep para análise estática de código e Trivy para varredura da imagem Docker em busca de vulnerabilidades críticas.

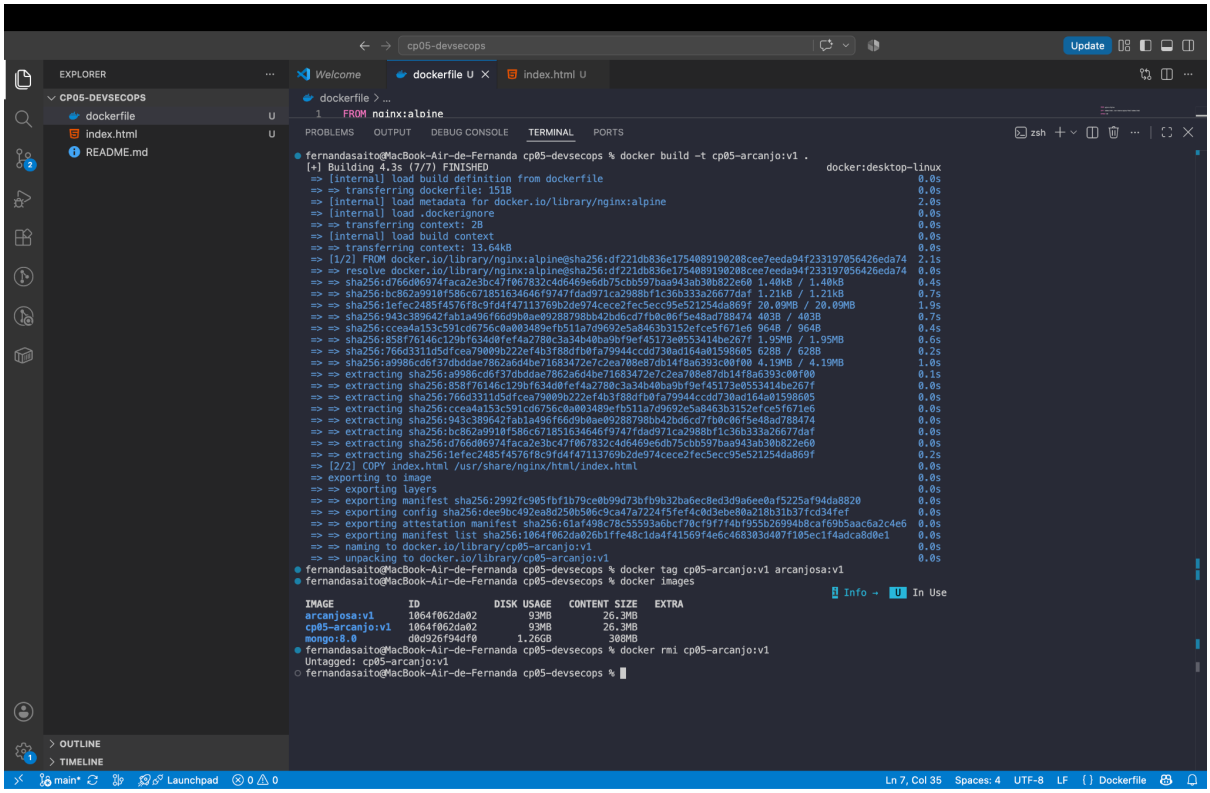
As simulações foram realizadas em branches de teste separadas da main. No Gitleaks, utilizou-se uma credencial AWS fictícia; no Semgrep, uma chamada à função eval(); e no Trivy, a inclusão temporária de uma versão vulnerável da biblioteca Log4j. Com isso, foi possível comparar execuções aprovadas e bloqueadas sem incorporar as alterações inseguras à versão principal.

Criação do Repositório com Dockerfile e arquivo HTML



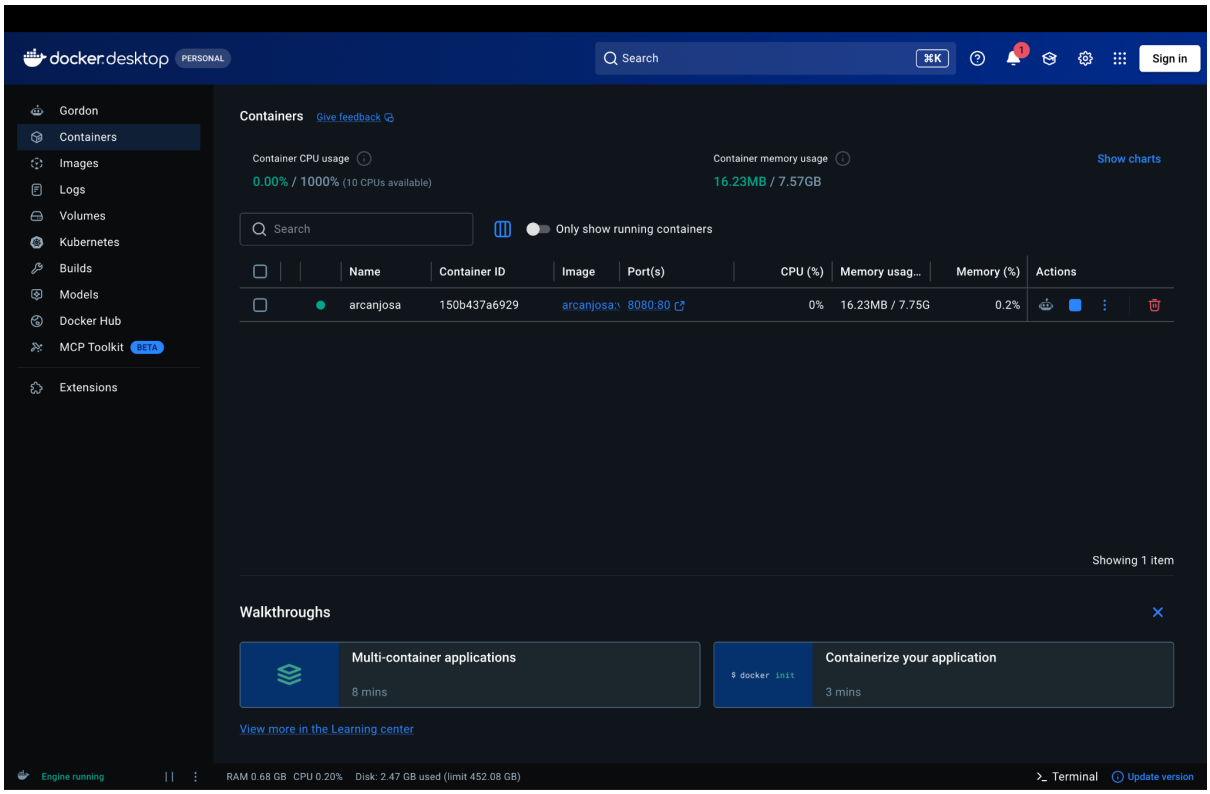
Criação da imagem docker



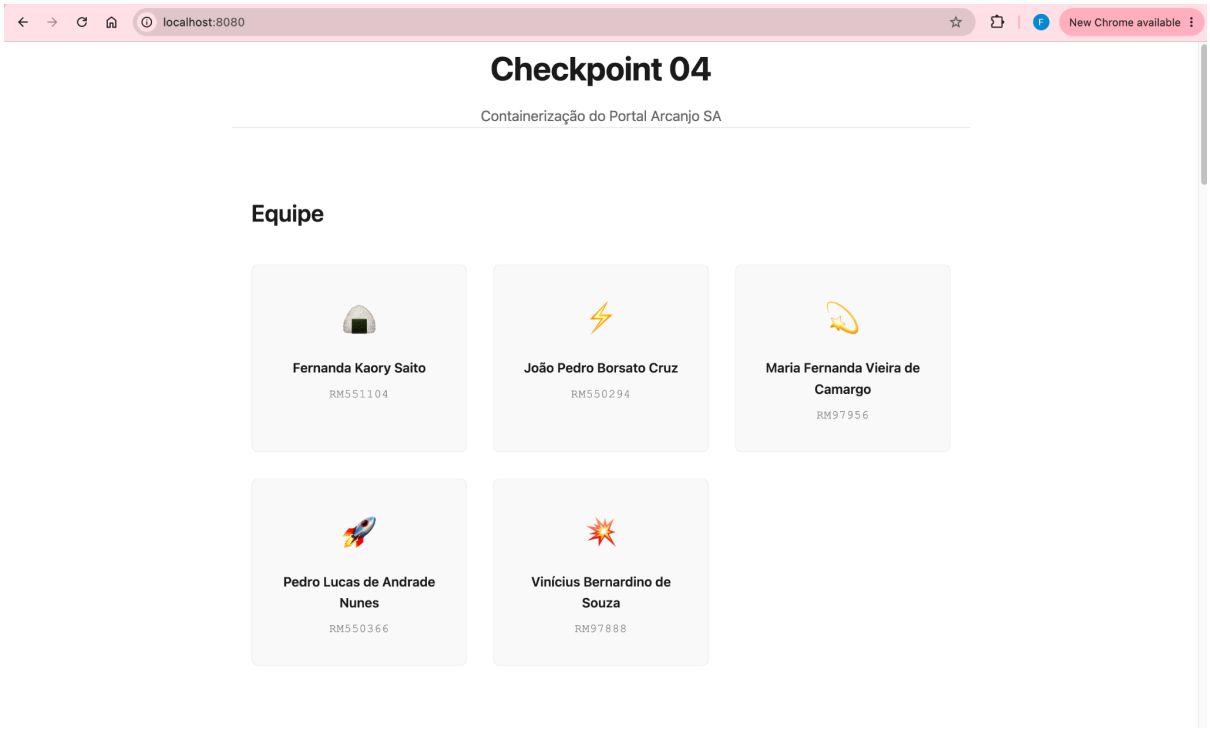


Criação do container

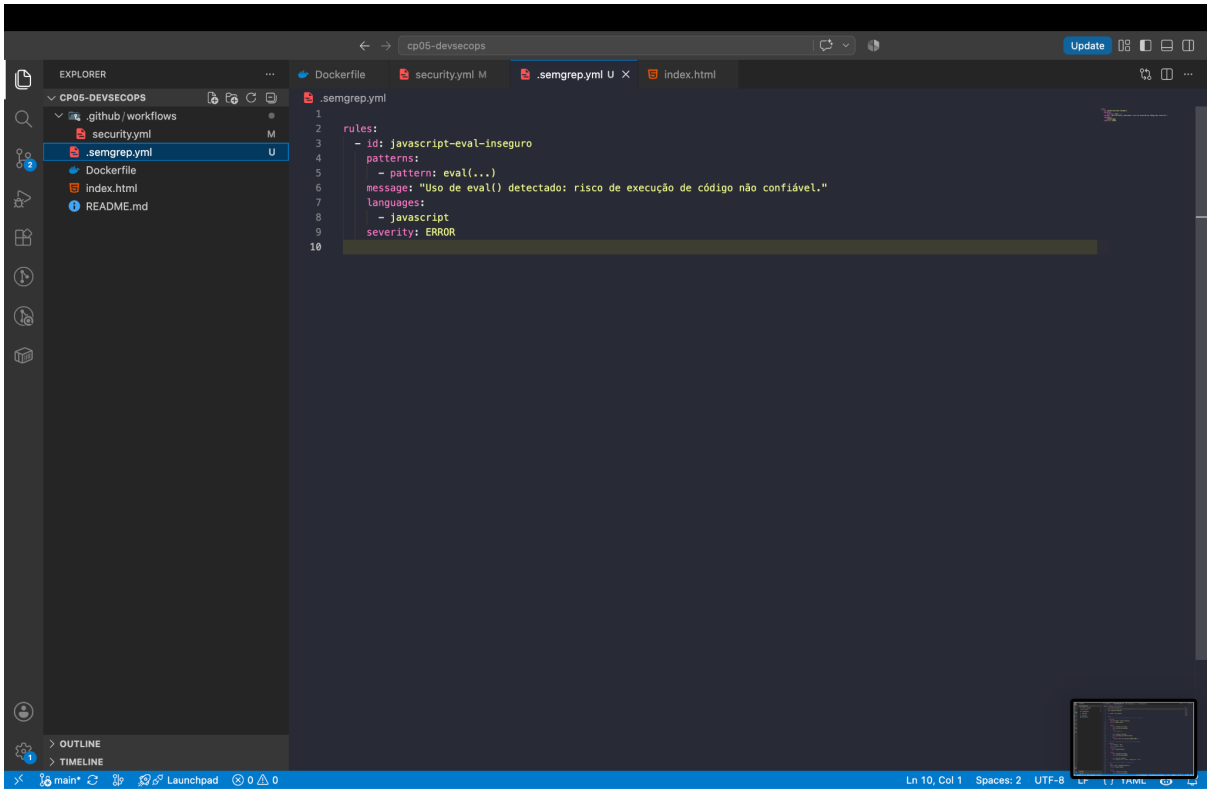
```
fernandasaito@MacBook-Air-de-Fernanda cp05-devsecops % docker images
                                                                                                    Info →  U  In Use
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
arcanjosa:v1   1064f062da02         93MB         26.3MB
mongo:8.0      d0d926f94df0       1.26GB          308MB
fernandasaito@MacBook-Air-de-Fernanda cp05-devsecops % docker run -d --name arcanjosa -p 8080:80 arcanjosa:v1
150b437a6929e0aa33f378a1df8612fb232b365511c4024c592fb5a79527ba75
fernandasaito@MacBook-Air-de-Fernanda cp05-devsecops % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS
                                                                                                                NAMES
150b437a6929   arcanjosa:v1   "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
                                                                                                                arcanjosa
fernandasaito@MacBook-Air-de-Fernanda cp05-devsecops %
```



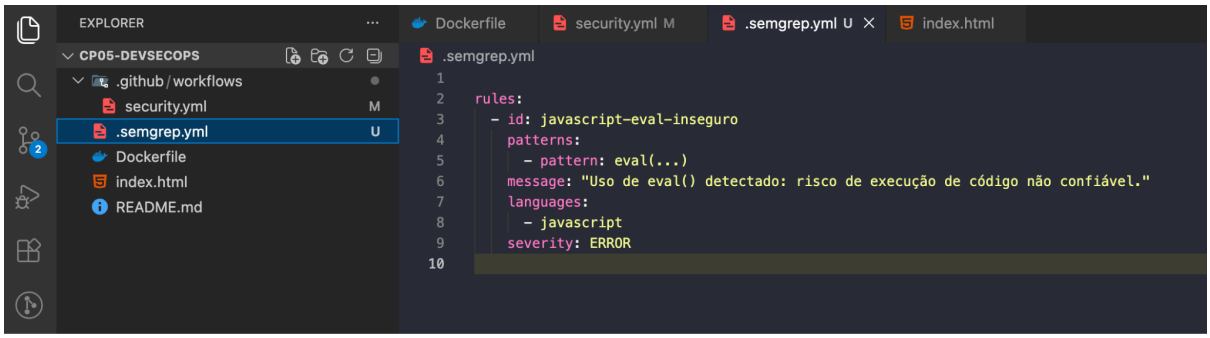
Site funcionando em http://localhost:8080



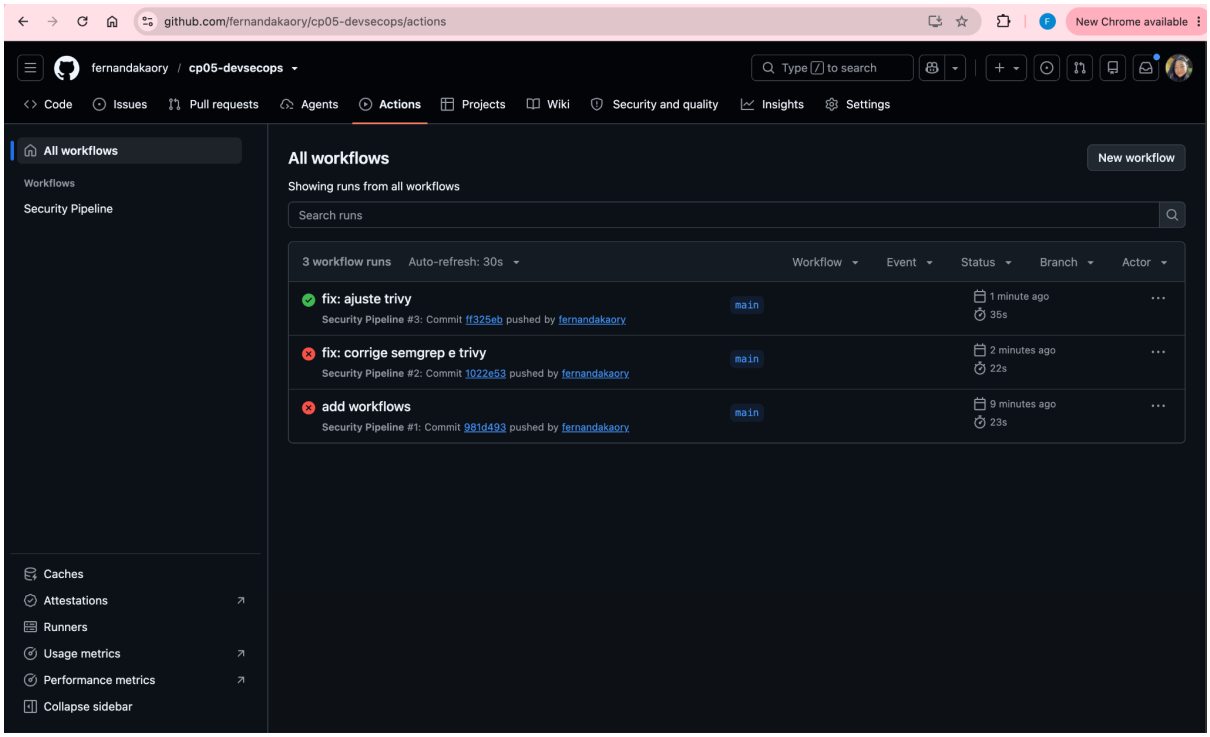
Criação do workflow com security.yml

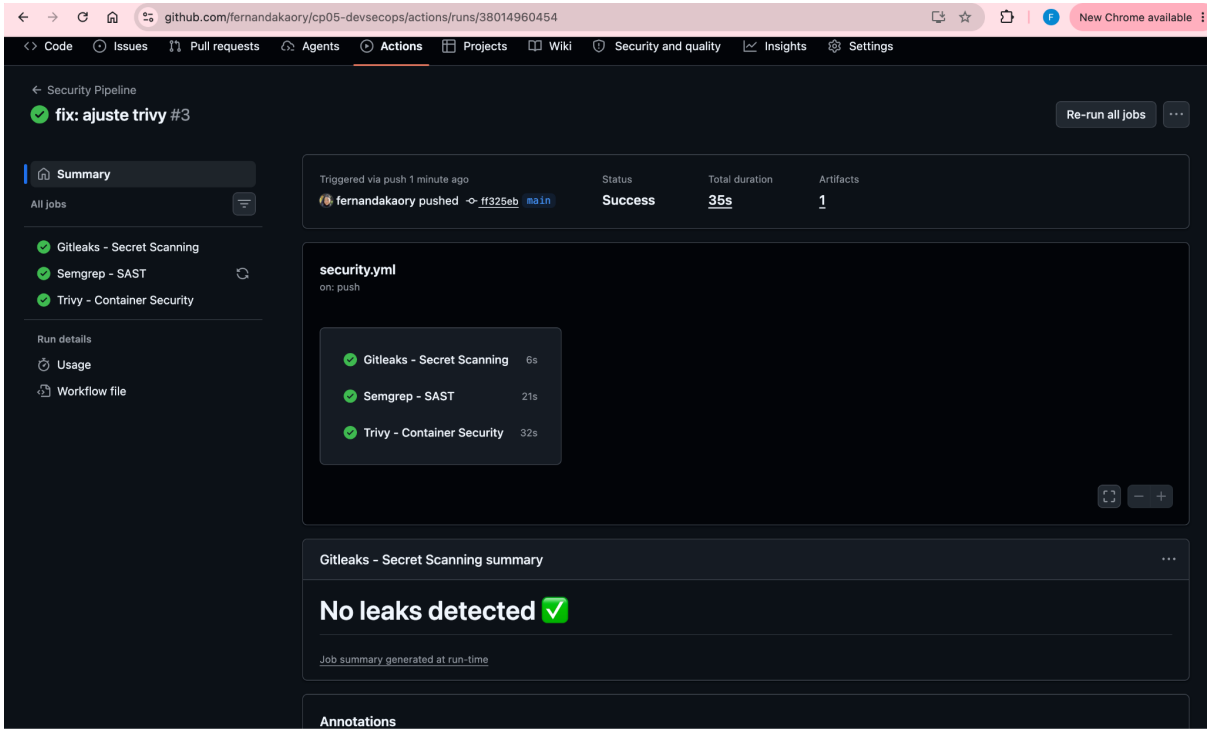


Criação de um semgrep.yml



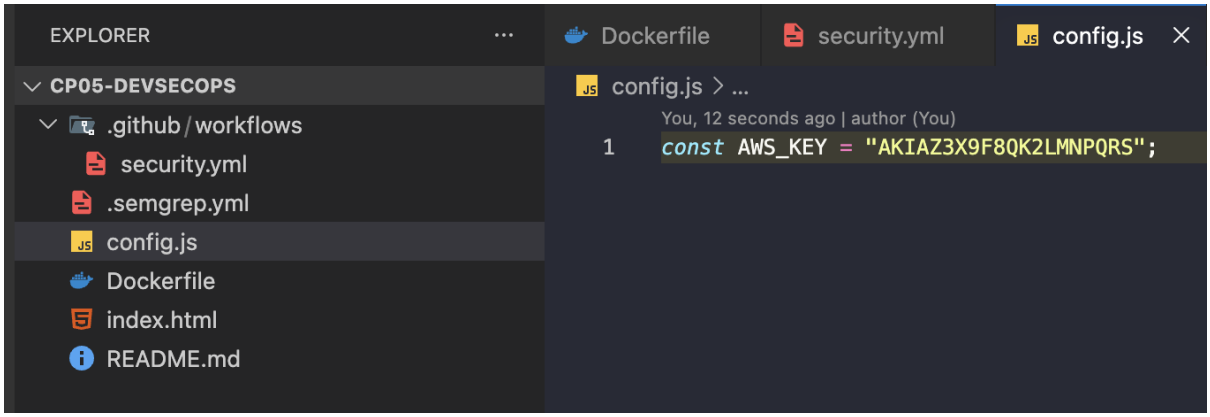
Subida do workflow



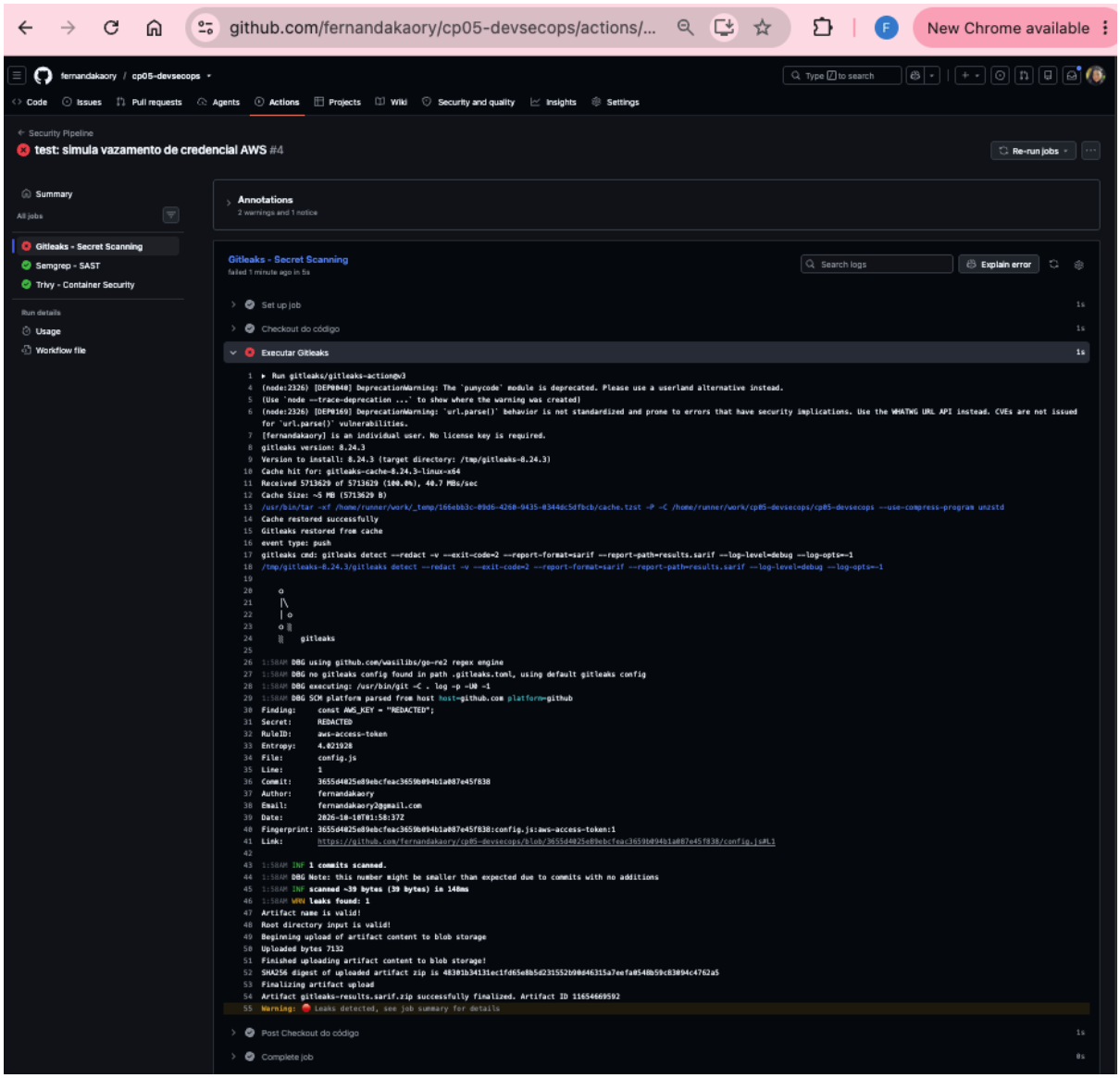


Criação de config.js para simular segredo vazado

Arquivo criado em branch teste-gitleaks



Git Leaks detectando e barrando o commit com segredo vazado



Teste 1 — Detecção de credenciais com Gitleaks

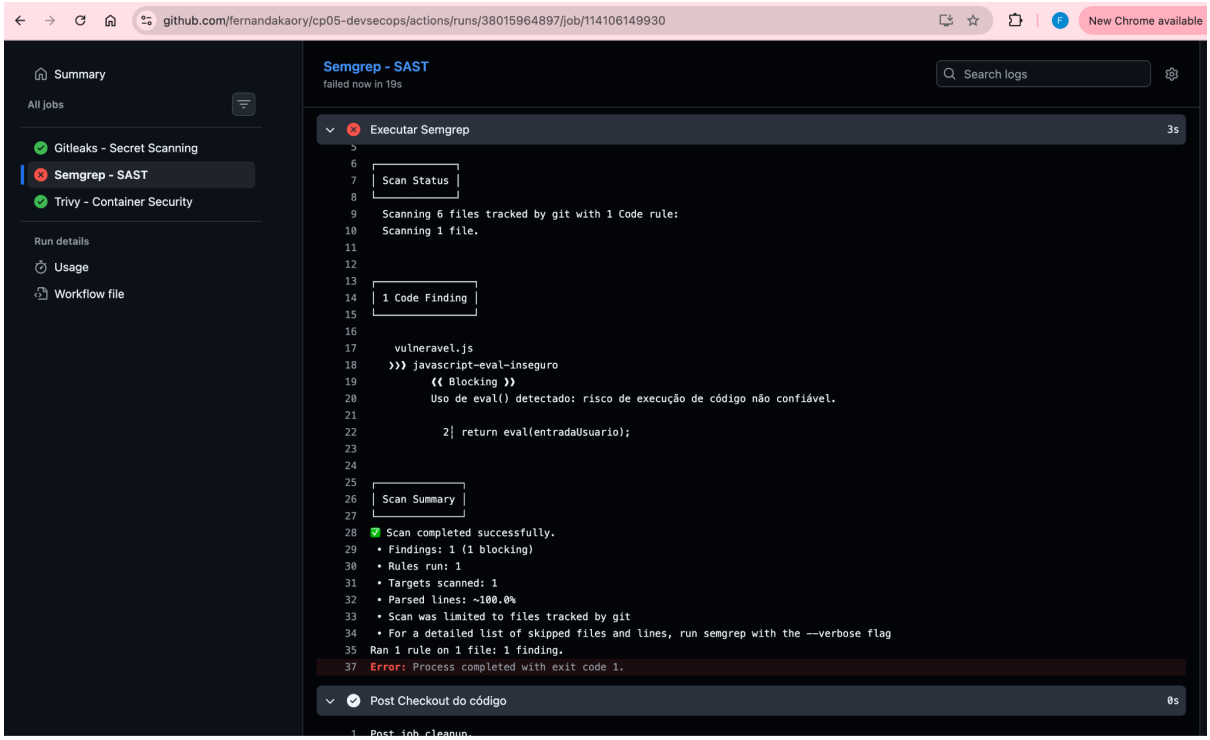
Para validar o funcionamento do Gitleaks, foi realizado um teste de segurança com a inserção proposital de uma credencial AWS fictícia no arquivo config.js. Após o commit e push para uma branch de testes, o pipeline do GitHub Actions executou a verificação de segredos e identificou o padrão de credencial exposto, interrompendo o job. Esse tipo de vulnerabilidade representa um risco porque credenciais reais publicadas em repositórios podem permitir acessos não autorizados a serviços e recursos em nuvem. O teste demonstrou a importância de automatizar a detecção de segredos durante o processo de desenvolvimento.

Criação de um arquivo vulneravel.js para teste com eval()

Testes feito na branch teste-semgrep



Semgrep detectando e barrando arquivo vulnerável com eval()

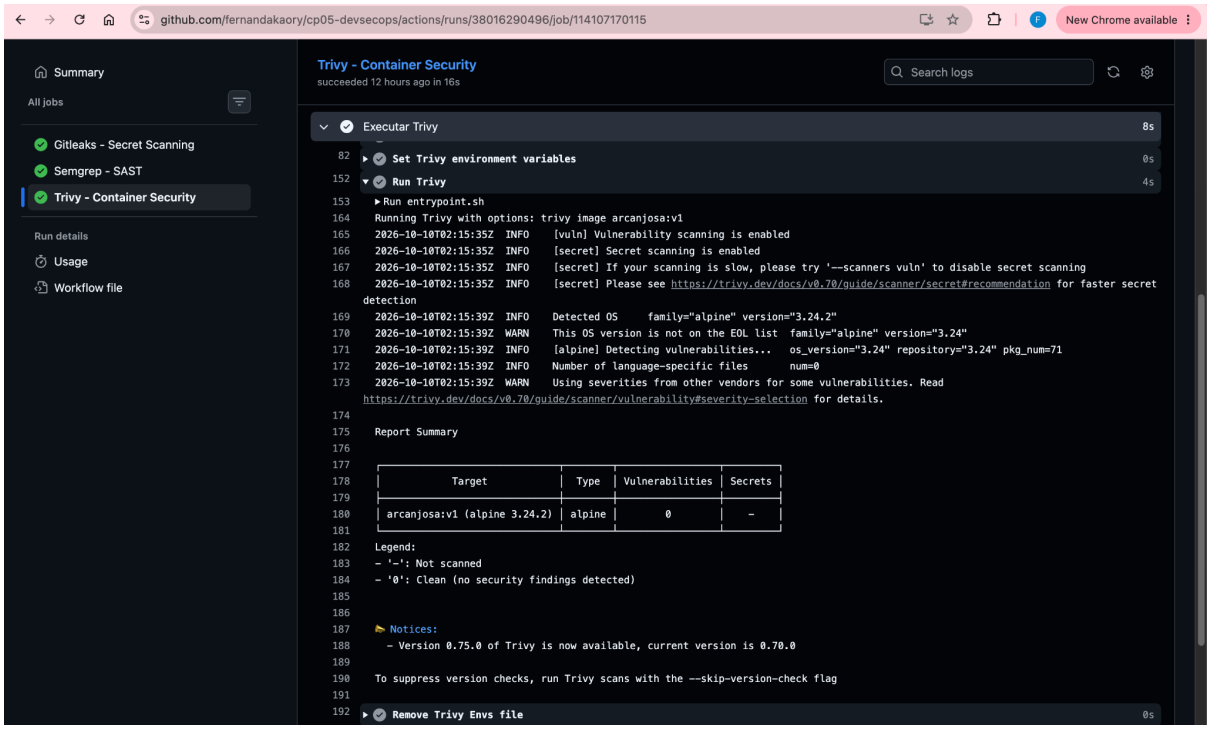


Teste 2 — Detecção de código vulnerável com Semgrep

Para validar a análise estática de segurança, foi inserido propositalmente um código JavaScript contendo a função eval() no arquivo vulneravel.js. Após o commit e push para uma branch de testes, o Semgrep executou a análise automatizada e identificou a utilização da função, interrompendo o job do pipeline. A vulnerabilidade representa um risco porque eval() permite interpretar e executar strings como código JavaScript, podendo causar a execução de comandos não autorizados quando recebe entradas não confiáveis. O teste demonstrou a importância da análise estática para identificar práticas inseguras antes da disponibilização da aplicação em produção.

Modificação temporária do Dockerfile para Trivy identificar vulnerabilidade

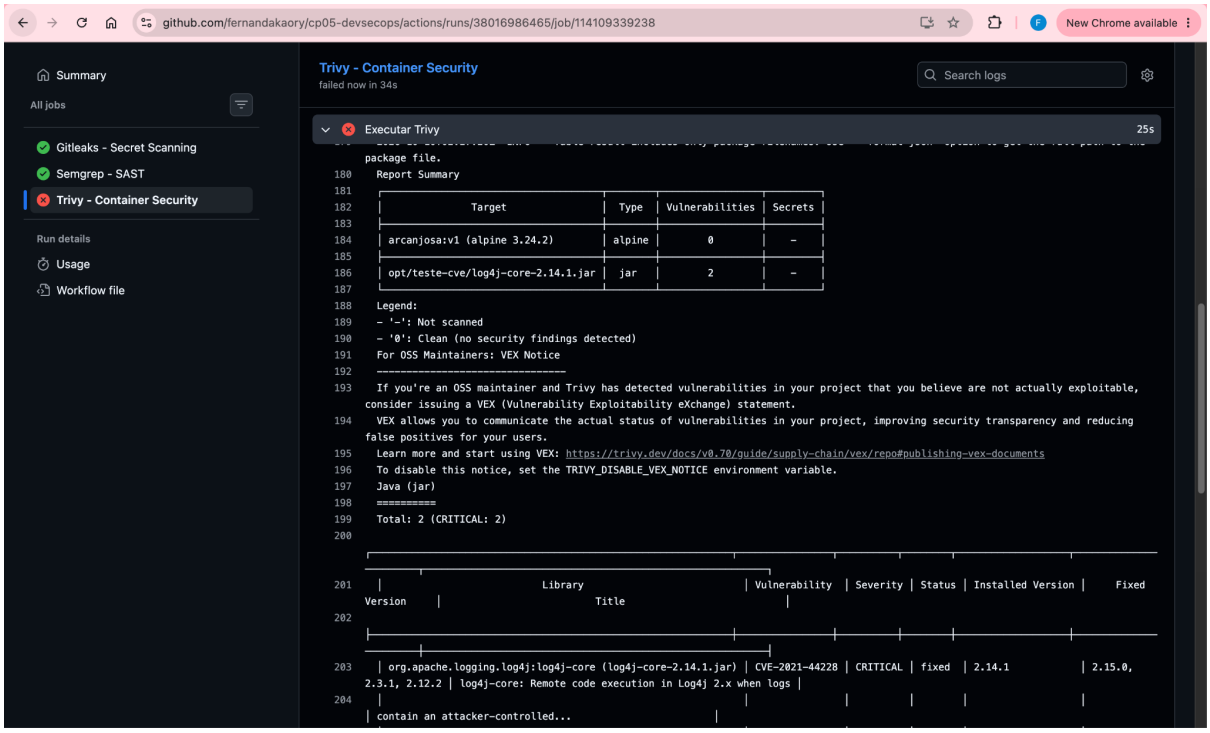
Trivy não apontando vulnerabilidades críticas

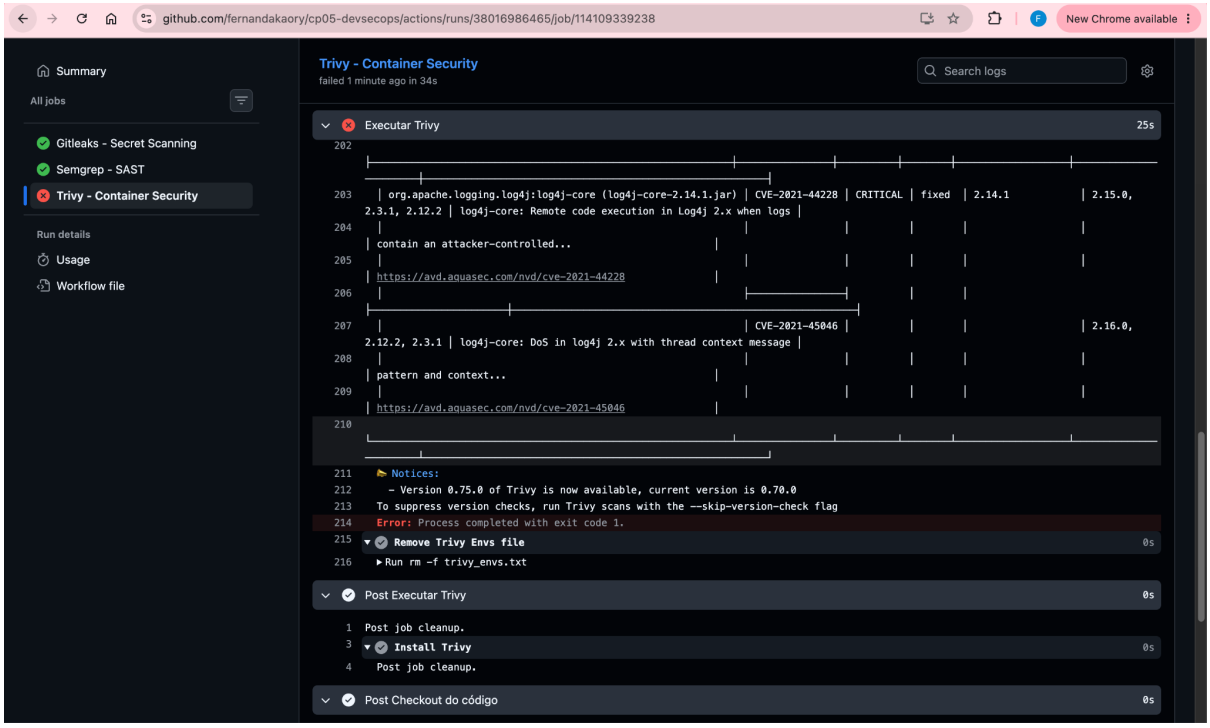


Teste feito na branch teste-trivy



Trivy identificando e barrando vulnerabilidade





Teste 3 — Validação de vulnerabilidades críticas com Trivy

Para validar o funcionamento da análise de segurança das imagens Docker, foram realizados dois testes com a ferramenta Trivy, integrada ao pipeline do GitHub Actions.

Inicialmente, a imagem arcanjosa:v1, baseada no Alpine Linux 3.24.2, foi analisada e não apresentou vulnerabilidades classificadas como críticas, permitindo a conclusão do job.

Em seguida, foi realizada uma simulação controlada por meio da inclusão da versão 2.14.1 da biblioteca Apache Log4j, associada à vulnerabilidade crítica CVE-2021-44228, em uma branch de testes. A ferramenta identificou a dependência vulnerável e interrompeu o job, demonstrando a capacidade do pipeline de bloquear imagens que não atendem aos critérios de segurança estabelecidos.

Esse procedimento evidencia a importância da análise automatizada de imagens Docker para prevenir a utilização de componentes com vulnerabilidades conhecidas durante o processo de entrega de software.

Conclusão

A atividade permitiu aplicar o conceito de DevSecOps por meio da integração de verificações de segurança ao pipeline de CI/CD do portal Arcanjosa. A containerização forneceu uma base reproduzível para a aplicação, enquanto o GitHub Actions automatizou a execução dos scanners a cada alteração enviada ao repositório.

Os testes demonstraram a detecção de uma credencial fictícia pelo Gitleaks, o bloqueio do uso de eval() pelo Semgrep e a identificação de vulnerabilidades críticas pelo Trivy após a inclusão de uma dependência vulnerável. A execução da imagem original também não apresentou vulnerabilidades críticas nas condições analisadas. Esses resultados evidenciam a importância de verificar tanto o código quanto as dependências e os artefatos de entrega.

Por fim, o projeto reforçou a prática de Shift Left, antecipando a identificação de riscos ainda nas etapas de desenvolvimento e integração. As ferramentas automatizadas auxiliam na prevenção de falhas, mas não substituem a revisão humana nem garantem, isoladamente, a segurança completa da aplicação.